Signal And Image Processing Lab

DR.nouri

Electrical Engineering Department

Ahmadreza Majlesara 400101861
Parsa Hatami 400100962

Lab 9

May 11, 2025

Prove equations

In the FCM method, the cost function for this method can be expressed as follows:

$$J_m = \sum_{i=1}^{D} \sum_{j=1}^{N} \mu_{ij}^m \|x_i - c_j\|^2$$

where D is the number of data points, N is the number of clusters, m is a positive constant used to control the fuzziness of the memberships, x_i represents the i-th data point, c_j is the center of cluster j, and μ_{ij} represents the degree of membership of data point x_i to cluster j. The sum of memberships for a single data point across all clusters is equal to one, i.e.,

$$\sum_{j=1}^{N} \mu_{ij} = 1 \quad \text{for } i = 1, 2, \ldots, D.$$

To minimize J_m with respect to both μ_{ij} and c_j for all i and j, the Expectation-Maximization algorithm is used. Assuming an initial value for μ_{ij} and keeping μ_{ij} fixed, the optimization of c_j is performed as follows:

$$c_j = \frac{\sum_{i=1}^{D} \mu_{ij}^m x_i}{\sum_{i=1}^{D} \mu_{ij}^m}.$$

Next, by keeping c_j fixed, the optimization of μ_{ij} is performed as:

$$\mu_{ij} = \frac{1}{\sum_{k=1}^{N} \left(\frac{\|x_i - c_j\|}{\|x_i - c_k\|} \right)^{\frac{2}{m-1}}}.$$

The cost function J_m is then computed. These two relations are repeated iteratively until the changes in J_m are negligible and it converges to a stable threshold.

solution

The cost function for the Fuzzy C-Means (FCM) algorithm is given by:

$$J_m = \sum_{i=1}^{D} \sum_{j=1}^{N} \mu_{ij}^m \|x_i - c_j\|^2,$$

where μ_{ij} is the membership degree of data point x_i in cluster c_j, and $m > 1$ is the fuzziness parameter.
To minimize J_m with respect to c_j, we take the partial derivative of J_m with respect to c_j and set it equal to zero:

$$\frac{\partial J_m}{\partial c_j} = \frac{\partial}{\partial c_j} \left(\sum_{i=1}^{D} \sum_{j=1}^{N} \mu_{ij}^m \|x_i - c_j\|^2 \right) = 0.$$

Since $\|x_i - c_j\|^2 = (x_i - c_j)^\top (x_i - c_j)$, we compute the derivative:

$$\frac{\partial}{\partial c_j}\|x_i - c_j\|^2 = \frac{\partial}{\partial c_j}\Big((x_i - c_j)^\top (x_i - c_j)\Big) = -2(x_i - c_j).$$

Substituting this into the expression for J_m:

$$\frac{\partial J_m}{\partial c_j} = \sum_{i=1}^{D} \mu_{ij}^m (-2)(x_i - c_j).$$

Setting this equal to zero gives:

$$\sum_{i=1}^{D} \mu_{ij}^m (x_i - c_j) = 0.$$

$$\sum_{i=1}^{D} \mu_{ij}^m x_i = c_j \sum_{i=1}^{D} \mu_{ij}^m.$$

Solving for c_j, we obtain the cluster center update equation:

$$c_j = \frac{\sum_{i=1}^{D} \mu_{ij}^m x_i}{\sum_{i=1}^{D} \mu_{ij}^m}.$$

To minimize J_m with respect to μ_{ij}, we impose the constraint:

$$\sum_{j=1}^{N} \mu_{ij} = 1, \quad \text{for all } i.$$

We use the method of Lagrange multipliers. Define the Lagrangian:

$$\mathscr{L} = \sum_{i=1}^{D}\sum_{j=1}^{N} \mu_{ij}^m \|x_i - c_j\|^2 + \sum_{i=1}^{D} \lambda_i \left(\sum_{j=1}^{N} \mu_{ij} - 1\right).$$

Take the partial derivative of $\mathscr{L}$ with respect to μ_{ij} and set it to zero:

$$\frac{\partial \mathscr{L}}{\partial \mu_{ij}} = m\mu_{ij}^{m-1}\|x_i - c_j\|^2 + \lambda_i = 0.$$

$$\mu_{ij}^{m-1}\|x_i - c_j\|^2 = -\frac{\lambda_i}{m}.$$

Using the constraint $\sum_{j=1}^{N} \mu_{ij} = 1$, we solve for μ_{ij}. First, express μ_{ij}:

$$\mu_{ij} = \left(\frac{1}{\|x_i - c_j\|^2}\right)^{\frac{1}{m-1}} \Bigg/ \sum_{k=1}^{N} \left(\frac{1}{\|x_i - c_k\|^2}\right)^{\frac{1}{m-1}}.$$

Simplify this to the final form:

$$\mu_{ij} = \frac{1}{\sum_{k=1}^{N} \left(\frac{\|x_i - c_j\|}{\|x_i - c_k\|}\right)^{\frac{2}{m-1}}}.$$

Question 1

Consider the image `thorax_t1.jpg` located in the folder `S3_Q1_utils`. The goal is to segment regions of tissues similar to fat and liver tissue (shapes 3-7 and 7-4).

First, identify the central pixel of these tissues by visual inspection. Then, using the intensity and brightness of the central pixel, determine appropriate thresholds for the brightness to start segmenting the surrounding pixels.

Perform the segmentation by iteratively moving to adjacent pixels and grouping the regions. Finally, display the segmented regions on the initial image as shown in the final output (shapes 3-7 and 7-4).

solution

segmentation for lungs :
To determine the central pixels, we first select two values by estimation based on visual inspection of the image. Then, by increasing and decreasing these values and observing the resulting changes in the output, we identify the most suitable central pixels. In this case, the values $[90, 95]$ and $[90, 175]$ are obtained.
Next, through trial and error, we find the appropriate values for the radius and margin. Here, the selected value is 30.
In the following step, we need to find an appropriate threshold for brightness. For this, by observing the brightness and darkness intensity of the components in the image and roughly analyzing several cells of the image matrix, an initial reasonable guess can be made. Then, using trial and error and observing the changes in the output, we adjust it to the best possible value. In this case, the chosen value is 50.

As can be seen in the output below, these values are appropriately selected, and the resulting output effectively identifies and separates the texture.

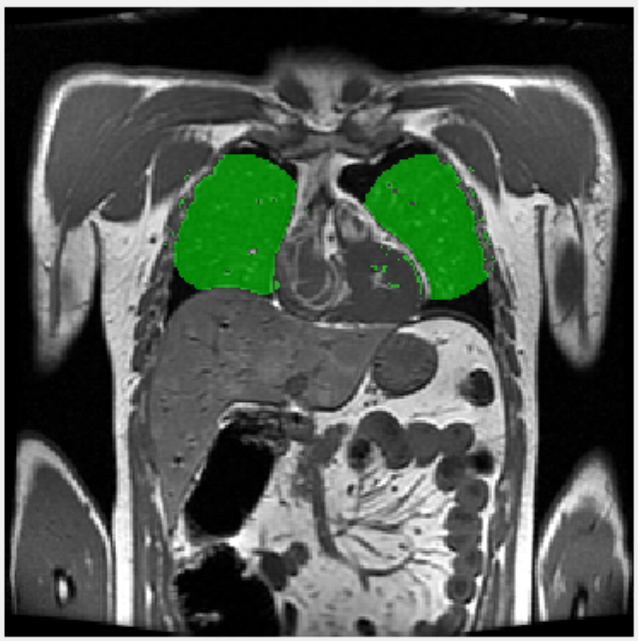

Figure 1. segmentation for tissues

segmentation for liver :
Referring to the shape of the liver, I used three circles to describe the area of segmentation. To determine the central pixels, we first select three values by estimation based on visual inspection of the image. Then, by increasing and decreasing these values and observing the resulting changes in the output, we identify the most suitable central pixels. In this case, the values $[180, 88]$, $[140, 88]$ and $[150, 130]$ are obtained. The estimation process for this part was more complicated than the previous part.
Next, through trial and error, we find the appropriate values for the radius and margin. Here, the selected value is 32.
In the following step, we need to find an appropriate threshold for brightness. For this, by observing the brightness and darkness intensity of the components in the image and roughly analyzing several cells of the image matrix, an initial reasonable guess can be made. Then, using trial and error and observing the changes in the output, we adjust it to the best possible value. In this case, we have several tissues beside, and then we should find three upper and lower limits for each area. Here are three limits for brightness thresholds for 3 areas: $[85, 135]$, $[180, 88]$ and $[180, 88]$
As can be seen in the output below, these values are appropriately selected, and the resulting output effectively identifies and separates the texture.

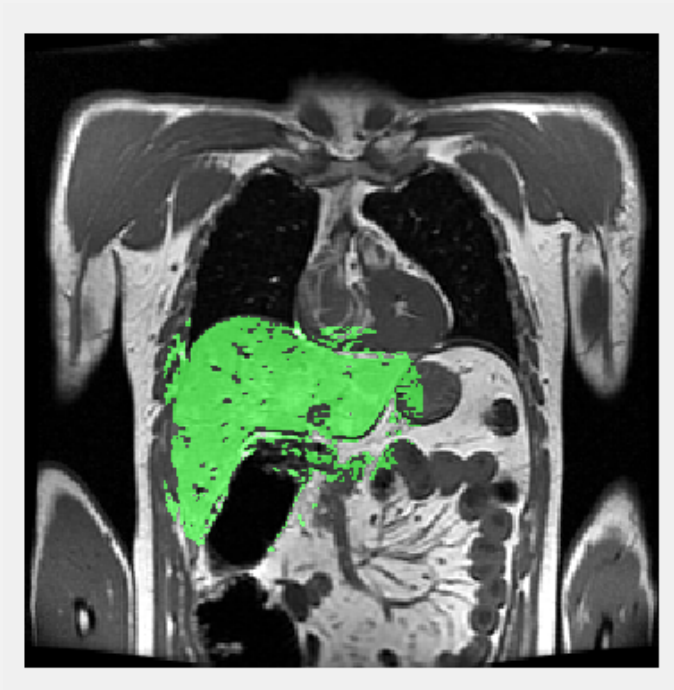

Figure 2. segmentation for tissues

```
%% section Q1 part1

image_path = "S3_Q1_utils/thorax_t1.jpg";
image = imread(image_path);

image = image(:, :, 1);

I = double(image);
```

```

imshow(I, []);
title('Input Image');

center1 = [90, 95];
center2 = [90, 175];
radius = 30;

flag = zeros(size(I));

[row, col] = size(I);
for i = 1:row
    for j = 1:col
        dist1 = sqrt((i - center1(1))^2 + (j - center1(2))^2);
        dist2 = sqrt((i - center2(1))^2 + (j - center2(2))^2);
        if (dist1 < radius || dist2 < radius) && I(i, j) <= 50
            flag(i, j) = 1;
        end
    end
end

Overlay(I, flag);

%% section Q1 part2

image_path = "S3_Q1_utils/thorax_t1.jpg";
image = imread(image_path);

image = image(:, :, 1);

I = double(image);

imshow(I, []);
title('Input Image');

center1 = [180, 88];
center2 = [140, 88];
center3 = [150, 130];

radius = 32;

flag = zeros(size(I));

[row, col] = size(I);
for i = 1:row
    for j = 1:col
        dist1 = sqrt((i - center1(1))^2 + (j - center1(2))^2);
        dist2 = sqrt((i - center2(1))^2 + (j - center2(2))^2);
        dist3 = sqrt((i - center3(1))^2 + (j - center3(2))^2);

        if (dist1 < radius) && (I(i, j) >= 85) && (I(i, j) <= 135)
            flag(i, j) = 1;
        end
        if (dist2 < radius) && (I(i, j) >= 70) && (I(i, j) <= 200)
            flag(i, j) = 1;
        end
        if (dist3 < radius) && (I(i, j) >= 70) && (I(i, j) <= 120)
            flag(i, j) = 1;
```

```
        end
    end
end

Overlay(I, flag);
```

Question 2

In this question, the goal is to produce the results of Figure 1-7 using the `kmeans` method.

To achieve this, read the three images `t1.jpg`, `t2.jpg`, and `pd.jpg` from the folder `S3_Q2_utils`, then concatenate them.

Next, consider the three-dimensional vectors of the pixels as their features. Use the `kmeans` command to cluster the data into 6 clusters, segment the region, and display the data as shown in Figure 1-7.

solution

In this part, we use MATLAB kmeans algorithm for dividing into 6 clusters.

Here is the output of clustering into 6 clusters using MATLAB kmeans :

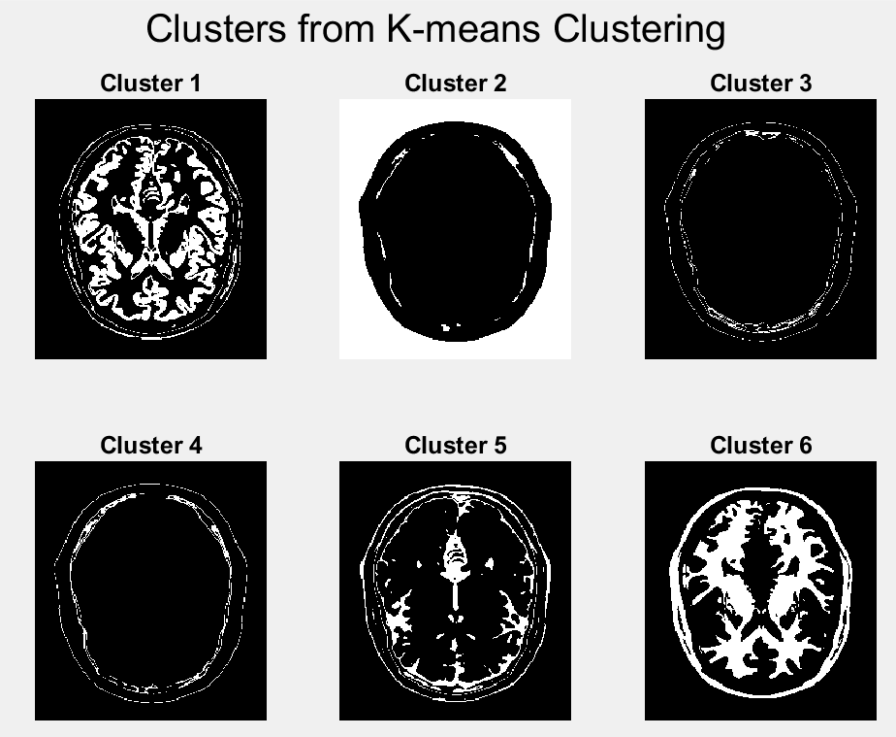

Figure 3. Clusters from K-means Clustering

The output of the K-means clustering method successfully segments the image into six distinct clusters. Each cluster isolates different regions of the brain's structures based on pixel intensity and spatial distribution.

For example, Cluster 1 and Cluster 6 effectively capture the main anatomical details, while Clusters 3 and 4 highlight the peripheral regions. The segmentation demonstrates the effectiveness of K-means in identifying meaningful patterns, with each cluster corresponding to a unique feature or intensity range in the image. This segmentation provides a good foundation for further analysis or region-specific processing tasks.

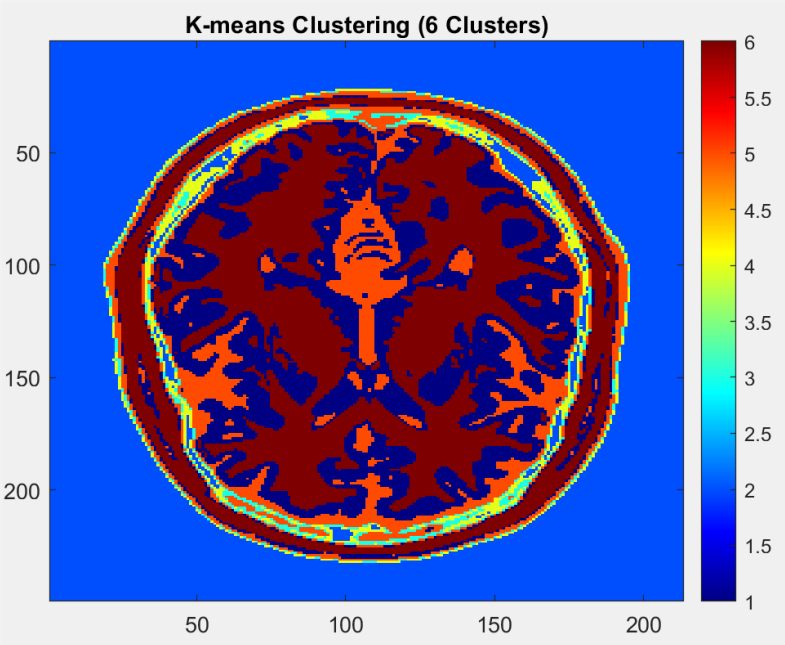

Figure 4. Clusters from K-means Clustering

```
%% section Q2

image_t1 = imread("S3_Q2_utils/t1.jpg");
image_t2 = imread("S3_Q2_utils/t2.jpg");
image_pd = imread("S3_Q2_utils/pd.jpg");

if size(image_t1, 3) == 3
    image_t1 = rgb2gray(image_t1);
end
if size(image_t2, 3) == 3
    image_t2 = rgb2gray(image_t2);
end
if size(image_pd, 3) == 3
    image_pd = rgb2gray(image_pd);
end

image_t1 = double(image_t1);
image_t2 = double(image_t2);
image_pd = double(image_pd);

[rows, cols] = size(image_t1);
features = [image_t1(:), image_t2(:), image_pd(:)];

num_clusters = 6;
[cluster_idx, cluster_centers] = kmeans(features, num_clusters, 'MaxIter',
```

```
    1000);

clustered_image = reshape(cluster_idx, rows, cols);

figure;
imagesc(clustered_image);
colormap('jet');
colorbar;
title('K-means Clustering (6 Clusters)');

imwrite(uint8(clustered_image * (255 / num_clusters)), 'clustered_output.jpg
    ');

figure;
for k = 1:num_clusters
    cluster_mask = (clustered_image == k);

    subplot(2, 3, k);
    imagesc(cluster_mask);
    colormap('gray');
    title(['Cluster ', num2str(k)]);
    axis off;
end

sgtitle('Clusters from K-means Clustering');
```

Question 3

Reimplement the results of Question 2 without using the `kmeans` function and by programming it from scratch. Compare your results with those obtained in Question 2. In fact, implement the `kmeans` function yourself.

solution

the result are shown in below figure:

Figure 5. result of the implemented function without using kmeans

The image, created with custom code, captures similar clusters but with less clarity and precision, likely due to simpler initialization and convergence criteria.

MATLAB's function is faster and better optimized, whereas the custom implementation, while functional, could benefit from improved initialization, tighter convergence thresholds, and normalization to achieve comparable results.

```
k = 6;
[rows, cols] = size(image_t1);
num_features = size(features, 1);
centers = features(randi(num_features, k, 1), :);
previous_centers = zeros(k, size(features, 2));
cluster_labels = zeros(num_features, 1);

while norm(centers - previous_centers, 'fro') > 0.1
    previous_centers = centers;


    for i = 1:num_features
        distances = vecnorm(features(i, :) - centers, 2, 2);
        [~, min_idx] = min(distances);
        cluster_labels(i) = min_idx;
    end

    for j = 1:k
        assigned_points = features(cluster_labels == j, :);
        if ~isempty(assigned_points)
            centers(j, :) = mean(assigned_points, 1);
        end
    end
end

cluster_reshaped = reshape(cluster_labels, rows, cols);

figure;
for i = 1:k
    binary_mask = (cluster_reshaped == i);
    J = zeros(rows, cols);
    J(binary_mask) = 1;
    subplot(2, 3, i);
    imshow(J, []);
    title(['Cluster ', num2str(i)]);
end
```

Question 4

Segment the clusters obtained in Question 2 using the `FCM` method. Use the `fcm` command appropriately, draw the resulting clusters, and compare them with the results of Question 2. To determine the final membership of each pixel to a cluster, use the maximum values of μ_{ij}.

solution

the output is shown in below figure:

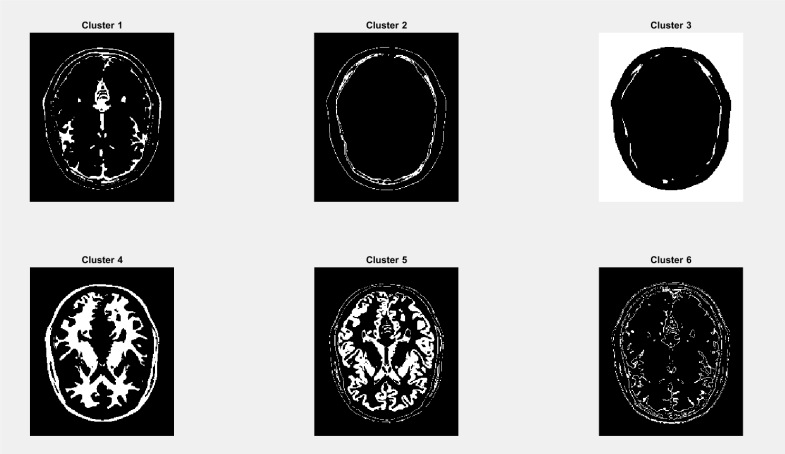

Figure 6. clusters created with fcm method

The image, processed with K-means, provides sharp and well-defined cluster boundaries, making it ideal for precise segmentation tasks. In contrast, the image processed with FCM shows smoother and more blended cluster transitions due to its fuzzy membership approach, which captures gradual changes but can blur finer details. K-means is better suited for strict region separation, while FCM is more appropriate for highlighting overlapping or gradient-like patterns.

```
[centers_fcm, U] = fcm(features, k);
[~, max_indices] = max(U, [], 1);
cluster_reshaped = reshape(max_indices, rows, cols);
for i = 1:k
    cluster_mask = (cluster_reshaped == i);
    J = zeros(rows, cols);
    J(cluster_mask) = 1;
    subplot(2, 3, i);
    imshow(J, []);
    title(['Cluster ', num2str(i)]);
end
```

Question 5

Research Question: Investigate the quality improvement of the `Region Growing` method.

solution

The `Region Growing` method is an intuitive image segmentation technique, but its quality depends on several factors, such as seed selection, thresholding, and noise handling.

There are a few key strategies that can significantly improve its performance. Firstly, the selection of seed points is critical. Poorly chosen seeds may lead to over-segmentation or missing important regions. To address this, automated methods can be used to detect seed points based on features such as intensity peaks or edge detection. Additionally, using multiple seed points for complex regions ensures better coverage and segmentation.

Secondly, adaptive thresholding can enhance the segmentation process. A fixed threshold may not perform well in all cases, especially for images with varying intensities. Instead, dynamic thresholds that adjust based on local intensity variations can provide more flexibility. In some cases, gradient-based thresholds, which incorporate edge information, can also help limit region growth to natural boundaries.

Noise is another challenge in `Region Growing`, as it can cause incorrect pixel inclusion. Preprocessing the image with filters, such as Gaussian or median filters, can help reduce noise before segmentation. Furthermore, post-processing techniques, such as morphological operations (e.g., dilation and erosion), can refine the segmented regions and remove noise artifacts.
Stopping criteria are also important to prevent over-segmentation. One approach is to use boundary detection techniques, such as Sobel or Canny edge detectors, to limit the growth to within true region boundaries. Another option is to set reasonable size constraints for the regions being grown.

Incorporating multiple features, such as texture or color, can further improve the method. For instance, texture features like local entropy or variance can help differentiate regions with similar intensities but different textures. For color images, using multi-channel data (e.g., RGB or HSV) can enhance the similarity criteria.

Advanced hybrid techniques can also improve `Region Growing`. For example, combining it with region merging or clustering methods, such as K-means or graph-based approaches, can produce smoother and more accurate segmentation results.

Improvements to `Region Growing` have been successfully applied in many fields. In medical imaging, adaptive thresholds and multi-feature criteria have improved segmentation accuracy for identifying tumors or organs. In satellite imagery, combining texture and gradient-based methods has led to better region separation in complex landscapes.

By refining seed selection, adopting adaptive thresholds, handling noise effectively, and exploring hybrid approaches, the quality of the `Region Growing` method can be significantly enhanced. These improvements make the method more robust, accurate, and applicable to a wide range of segmentation tasks.

Question 6

Research Question: Investigate advanced region segmentation methods such as `Snake` and `Level-set`. Execute the demo provided in the `Snakes_demo` folder, briefly explain its process, and include the results in your report.

solution

the results are shown in below figures:

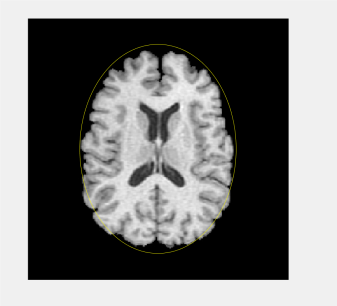

Figure 7. inital image

Figure 8. snake method output

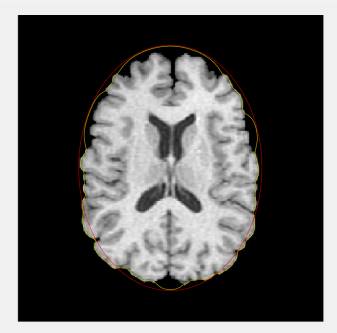

Figure 9. both images in 1 plot

The Snake method, also known as active contouring, is an advanced image segmentation technique that iteratively evolves a curve to fit object boundaries. The process involves initializing a contour (red line) near the desired boundary, which then adapts to the edges of the object based on image gradients and predefined energy functions. The method balances internal forces, ensuring smoothness, and external forces, driving the curve toward edges.

In the provided demo, the red curve was the initial contour, and after several iterations, the active contour (yellow line) converged to align with the brain's outer boundary, effectively segmenting the region.
The Level-set method on the other hand is a powerful segmentation technique that represents contours implicitly as the zero level of a higher-dimensional function. Unlike Snakes, it handles complex shapes and topological changes, like merging or splitting, without explicit reinitialization. The contour evolves by solving partial differential equations based on internal forces for smoothness and external forces to attract it to object boundaries. This makes the Level-set method more robust for segmenting noisy, irregular, or discontinuous boundaries, particularly in medical imaging, where structures often have intricate and ambiguous shapes.